ptt-tmux

Push-to-talk (toggle-to-talk) voice input for tmux. Speak into a terminal pane and have your words typed in live as you say them, powered by Parakeet running locally via MLX.

Setup

Dependencies are already installed in `.venv` (`parakeet-mlx`, `sounddevice`, `numpy`). If you need to reinstall:

```
cd ptt-tmux
python3 -m venv .venv
.venv/bin/pip install parakeet-mlx sounddevice numpy
```

Try it out

1. Start the daemon (first run downloads the model, then loads it — takes a bit):

   ```
   cd ptt-tmux
   .venv/bin/python daemon.py
   ```

   Leave this running in its own terminal/pane and watch its log output (`[ptt] ...`) for status.

2. In another pane, find your pane id:

   ```
   tmux display-message -p '#{pane_id}'
   ```

3. Toggle recording on, targeting that pane:

   ```
   .venv/bin/python toggle.py %<pane-id>
   ```

   Start speaking. Words should appear typed into the target pane within about a second of saying them.

4. Run the same command again to stop recording:

   ```
   .venv/bin/python toggle.py %<pane-id>
   ```

Selecting an input device

By default recording uses the system default microphone. To use a different input device, list available devices and pass one along when toggling on:

```
.venv/bin/python toggle.py --list-devices
.venv/bin/python toggle.py %<pane-id> 2          # by index
.venv/bin/python toggle.py %<pane-id> "MacBook Pro Microphone"  # by name
```

The device only needs to be specified on the toggle-on call; toggle-off just needs the pane id.

Volume threshold

While recording, each poll first checks the RMS volume of the audio captured since the last poll. If it's below `VOLUME_THRESHOLD` (default `0.01`, float32 scale), that pass is skipped and the model isn't invoked — cuts wasted CPU during pauses/silence in dictation. The final pass on toggle-off always runs regardless of volume, so trailing quiet words aren't dropped. Override the default with:

```
PTT_VOLUME_THRESHOLD=0.02 .venv/bin/python daemon.py
```

Wiring up a tmux key binding

Add to `~/.tmux.conf` so a single key toggles recording for whichever pane is focused (example binds it to `prefix + v`):

```
bind-key v run-shell "~/GenAI/genai-workspace/ptt-tmux/.venv/bin/python
↪  ~/GenAI/genai-workspace/ptt-tmux/toggle.py #{pane_id}"
```

Reload with `tmux source-file ~/.tmux.conf`, then press `prefix + v` to start/stop recording in the active pane.

Platform support

macOS only (Apple Silicon). Transcription runs via MLX, which only supports Apple Silicon — there's no Linux (or Intel Mac) equivalent today. Porting to Linux would mean swapping the MLX/Parakeet-MLX model loading in `daemon.py` for a CUDA/CPU-backed ASR stack (e.g. faster-whisper or the original PyTorch Parakeet checkpoint); the socket/tmux/toggle plumbing around it is otherwise platform-agnostic.

Functional requirements

1. Live input — words are typed into the target pane as they're spoken, not only after recording stops.
2. Fully local — no network calls; transcription runs on-device via MLX.
3. Low CPU / memory footprint — the daemon should stay lightweight both while idle and across long recording sessions.
4. Visual recording indicator — the tmux pane currently being recorded should show a red border, so it's obvious at a glance which pane is live and whether recording is still on.

How it works

- `daemon.py` loads the Parakeet model once and listens on a unix socket (`~/.ptt-tmux.sock`). While recording is active it re-transcribes the audio-so-far roughly once a second and types only the newly recognized words into the target tmux pane via `tmux send-keys`.
- `toggle.py` is a tiny client: it sends `toggle <pane-id>` to the daemon. First call starts recording, second call stops it.

Notes / current limitations

- This is a first working pass, not optimized: transcription re-runs on the full audio buffer every poll interval (~1s), so latency and CPU usage grow with utterance length. Fine for short dictation; a longer session would want proper streaming/chunked decoding.
- Only one recording session at a time (single global daemon state).
- The daemon must already be running before you call `toggle.py` (`ping` command exists for a basic liveness check if needed).

Known bugs / planned improvements

- **Silence gaps pick up background noise.** Because each poll re-transcribes the whole growing buffer, a pause in speech leaves a stretch of room tone/background noise sitting in that buffer; when speech resumes, the model can hallucinate words from that noise. Fix: mute (zero out) low-RMS windows of the buffer before handing it to the model, instead of passing the raw audio through.
- **Spurious full stop on audio cuts.** The model sometimes appends a trailing "." right at a silence cut, which gets typed even though it isn't really the end of a sentence. Fix: hold back a trailing lone "." until it's confirmed by a later poll or the final stop pass, instead of injecting it immediately.
- **Pane border reverts to green when clicked.** `pane-border-style` only affects the *inactive* border; tmux uses the separate `pane-active-border-style` for whichever pane currently has focus, so clicking

the recording pane makes it "active" and its border falls back to the default (green) style. Fix: set `pane-active-border-style fg=red` too while recording, and clear both on stop.

- **Retroactive cleanup via Ollama for long input.** For long utterances, if a local Ollama server is running, send the final transcript to it after stop to clean up punctuation/grammar/mis-transcriptions, then retype the corrected text in place of the raw transcript.